

# VulnCastle Lab Engineering Documentation

## Infrastructure Architecture Configuration Fix Log

Target Platform: Kali Linux (Host) / Ubuntu 22.04 (Container Backend)

| Component Impacted | Root Cause                                          | Operational Status                |
|--------------------|-----------------------------------------------------|-----------------------------------|
| Base Dockerfile    | Duplicate system user creation conflict ('backup')  | Resolved via usermod              |
| Service Entrypoint | Unsupported wildcards & missing dynamic directories | Resolved via exact string mapping |
| Nginx Config       | Socket naming version mismatch (generic vs 8.1)     | Resolved via exact path alignment |

## 1. Multi-User Creation Crash (Base Image Build)

### Technical Analysis & Root Cause:

During the primary compilation phase (docker build -t vulncastle-base:latest ./lab/base/), the build engine terminated with Exit Code: 9 at the user creation step. The underlying ubuntu:22.04 base distribution image natively provisions a system-level utility account named 'backup' during standard OS construction. The deployment instruction attempted to run a generic sequential installation sequence containing 'useradd -m -s /bin/bash backup'. Linux operating systems strictly prohibit the creation of duplicate accounts under identical identifiers, causing the compilation to abort instantly.

### Remediated Configuration (Dockerfile):

```
RUN useradd -m -s /bin/bash sysadmin && echo "sysadmin:admin123" | chpasswd \  
&& useradd -m -s /bin/bash devuser && echo "devuser:devuser2024" | chpasswd \  
&& usermod -s /bin/bash backup && echo "backup:backup" | chpasswd \  
&& useradd -m -s /bin/bash deploy && echo "deploy:deploy!" | chpasswd
```

## 2. Silent Application Failure (Container Initialization Script)

### Technical Analysis & Root Cause:

Upon launching the environment, external web clients encountered an unresolvable 502 Bad Gateway error at `http://localhost:8080`. Diagnostic validation inside the container (`ps aux`) revealed that while the Nginx master routing processes were operational, the underlying backend processor (PHP-FPM) was completely missing from the process tree.

The repository's service instantiation script contained a wildcard declaration (`service php*-fpm start`). The native Linux service framework expects a strictly defined service literal and treats variable wildcards as unresolvable expressions, failing silently. Furthermore, modern Debian/Ubuntu PHP-FPM architectures require a persistent process environment run-path (`/run/php/`) to establish operational sockets; Docker container lifecycles launch with empty dynamic file trees, which crashes the initialization of the runtime daemon.

### Remediated Configuration (start.sh):

```
#!/bin/bash
mkdir -p /run/php
service php8.1-fpm start
service cron start
/usr/sbin/sshd
nginx -g "daemon off;■■■"
```

## 3. Communication Socket Mismatch (Nginx Configuration)

### Technical Analysis & Root Cause:

Following the verification that the PHP-FPM application daemon was executing correctly inside the runtime environment, the container still returned a 502 Bad Gateway error. Analysis of the web server engine logs (`tail -n 20 /var/log/nginx/error.log`) revealed a socket naming version mismatch. The project configuration file directed the proxy engine to query a generic path (`/run/php/php-fpm.sock`). However, the compiled system environment provisions an isolated version-controlled socket named `/run/php/php8.1-fpm.sock`. Because the path names did not correlate, Nginx could not establish an internal link to process dynamic scripting instructions.

### Remediated Configuration (nginx.conf):

```
location ~ \.php$ {
include snippets/fastcgi-php.conf;
fastcgi_pass unix:/run/php/php8.1-fpm.sock;
}
```